影像處理
--Final Project

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/12/31

1. **code**

```
import cv2
import torch
from ultralytics import YOLO
from collections import defaultdict

input_filename = "person_dog.mp4"
output_filename = "person_dog_314512065.mp4"
target_classes = [0, 16]  # person, dog
student_id = "314512065"
model_weights = "yolov8x.pt"

CLASS_COLORS = {
    0: (255, 191, 0),   # person
    16: (0, 140, 255)   # dog
}
DEFAULT_COLOR = (0, 255, 0)

IMGSZ = 1280
CONF = 0.35
IOU = 0.55
MIN_BOX_AREA = 900
TRACKER_YAML = "bytetrack.yaml"

def process_video_custom():
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model = YOLO(model_weights)
    model.to(device)
    class_names = model.names

    cap = cv2.VideoCapture(input_filename)
    if not cap.isOpened():
        print("Failed to open video:", input_filename)
        return

    width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
```

```
    fps = cap.get(cv2.CAP_PROP_FPS) or 30.0

    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    out = cv2.VideoWriter(output_filename, fourcc, fps, (width, height))

    my_font = cv2.FONT_HERSHEY_TRIPLEX
    font_scale = 1.0
    text_x = 30
    base_text_y = 60
    line_height = 50

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        results = model.track(
            frame,
            persist=True,
            tracker=TRACKER_YAML,
            classes=target_classes,
            conf=CONF,
            iou=IOU,
            imgsz=IMGSZ,
            verbose=False
        )

        result = results[0]
        boxes = result.boxes

        cls_to_ids = defaultdict(set)

        if boxes is not None and len(boxes) > 0:
            for box in boxes:
                cls_id = int(box.cls[0])
                conf = float(box.conf[0])

                x1, y1, x2, y2 = map(int, box.xyxy[0])
```

```
                if (x2 - x1) * (y2 - y1) < MIN_BOX_AREA:
                    continue

                track_id = None
                if box.id is not None:
                    track_id = int(box.id[0])
                    cls_to_ids[cls_id].add(track_id)

                color = CLASS_COLORS.get(cls_id, DEFAULT_COLOR)
                cv2.rectangle(frame, (x1, y1), (x2, y2), color, 3)

                name = class_names.get(cls_id, str(cls_id)) if
isinstance(class_names, dict) else class_names[cls_id]
                if track_id is not None:
                    label = f"{name} {conf:.2f} ID:{track_id}"
                else:
                    label = f"{name} {conf:.2f}"

                (w, h), _ = cv2.getTextSize(label,
cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
                y_top = max(0, y1 - 22)
                cv2.rectangle(frame, (x1, y_top), (x1 + w, y_top + 22),
color, -1)
                cv2.putText(frame, label, (x1, y_top + 16),
                            cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255, 255,
255), 2)

        text_y = base_text_y

        cv2.putText(frame, f"ID: {student_id}", (text_x + 2, text_y +
2),
                    my_font, font_scale, (50, 50, 50), 2)
        cv2.putText(frame, f"ID: {student_id}", (text_x, text_y),
                    my_font, font_scale, (0, 255, 255), 2)
        text_y += line_height

        if len(cls_to_ids) > 0:
```

```
            for cls_id in sorted(cls_to_ids.keys()):
                name = class_names.get(cls_id, str(cls_id)) if
isinstance(class_names, dict) else class_names[cls_id]
                count = len(cls_to_ids[cls_id])
                obj_color = CLASS_COLORS.get(cls_id, (255, 255, 255))
                text_str = f"{name} : {count}"

                cv2.putText(frame, text_str, (text_x + 2, text_y + 2),
                            my_font, font_scale, (50, 50, 50), 2)
                cv2.putText(frame, text_str, (text_x, text_y),
                            my_font, font_scale, obj_color, 2)
                text_y += line_height

        out.write(frame)

    cap.release()
    out.release()
    print("Saved:", output_filename)

if __name__ == "__main__":
    process_video_custom()
```

2. **方法說明：**

程式以 YOLOv8x 為核心模型，結合 ByteTrack 進行影片中人與狗之物件偵測與跨影格追蹤。系統首先根據硬體環境自動選擇 GPU 或 CPU 作為推論裝置，並以逐幀方式讀取輸入影片影像。每一影格會送入 YOLOv8 模型進行推論，僅針對 person 與 dog 類別進行偵測，以降低不必要的計算量。

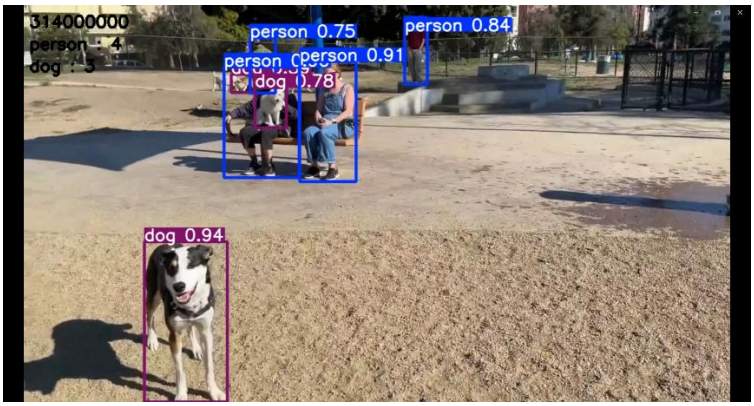
同時透過設定較高的輸入解析度（imgsz）與適當的信心值與 IoU 門檻，提高對遠距離與小型目標的辨識能力。為解決單純逐幀偵測易造成目標閃爍與重複計數的問題，我進一步整合 ByteTrack 追蹤機制，利用追蹤 ID 維持物件在跨影格間的一致性，並以追蹤 ID 為基礎進行唯一目標數量統計。此外，程式透過最小邊界框面積過濾機制移除尺寸過小的偽偵測框，以降低假陽性影響。

最終，系統將物件的邊界框、類別名稱、信心值與追蹤 ID 視覺化顯示於影像中，並輸出為一支兼具即時性與穩定性的影片物件偵測結果。

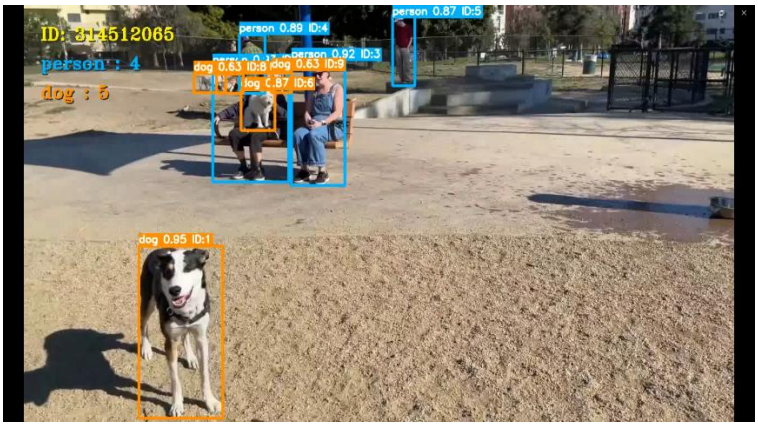
3. **加分說明：**

(1)

Baseline frame:

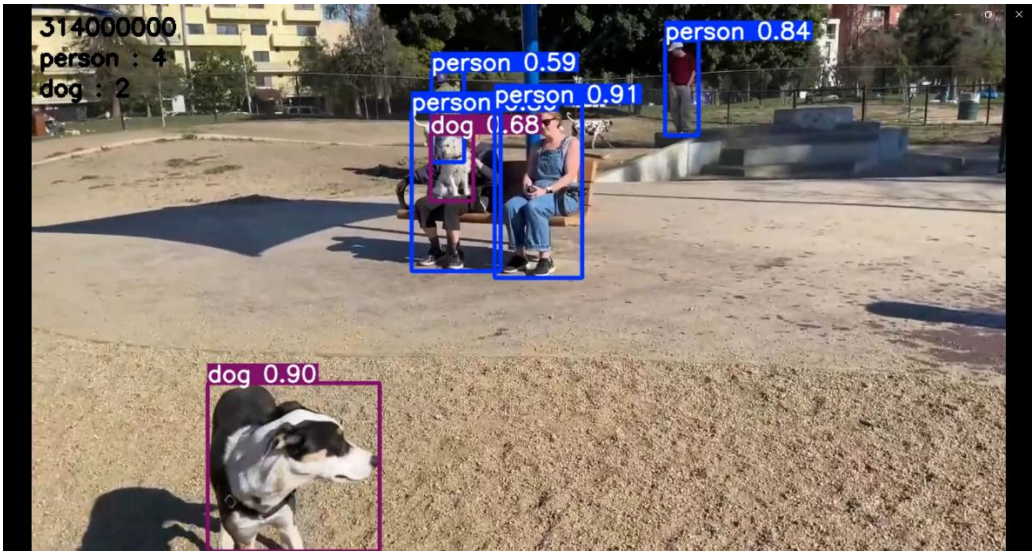


My video frame:

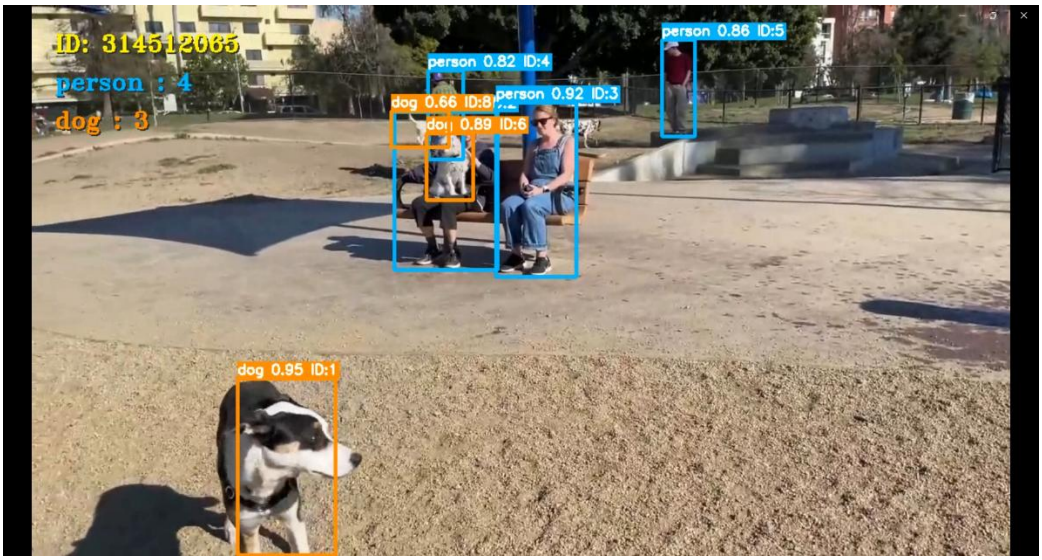


(2)

Baseline frame:

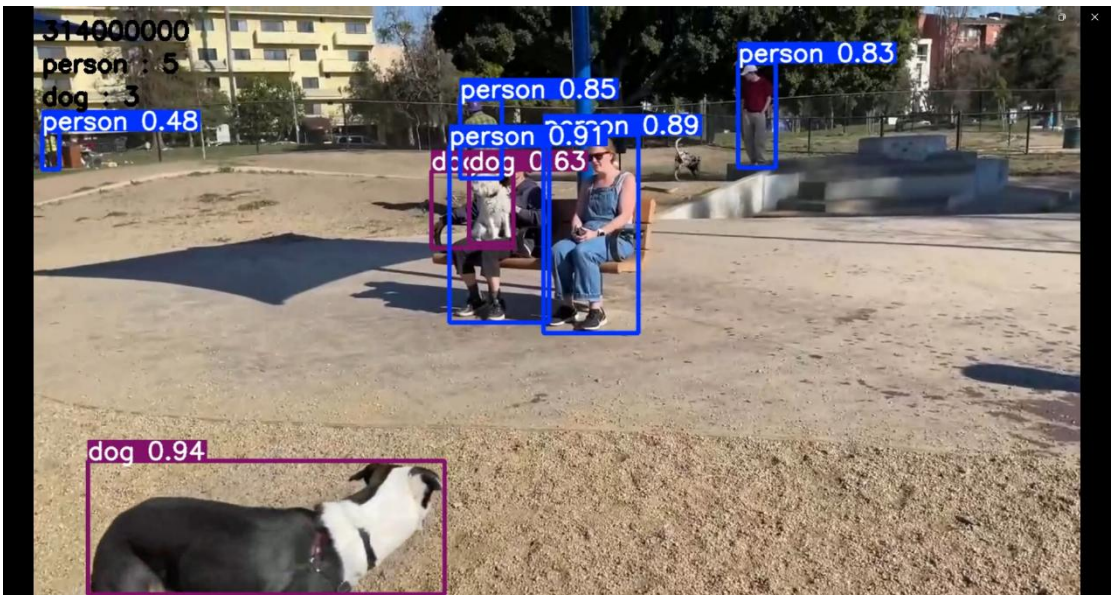


My video frame:

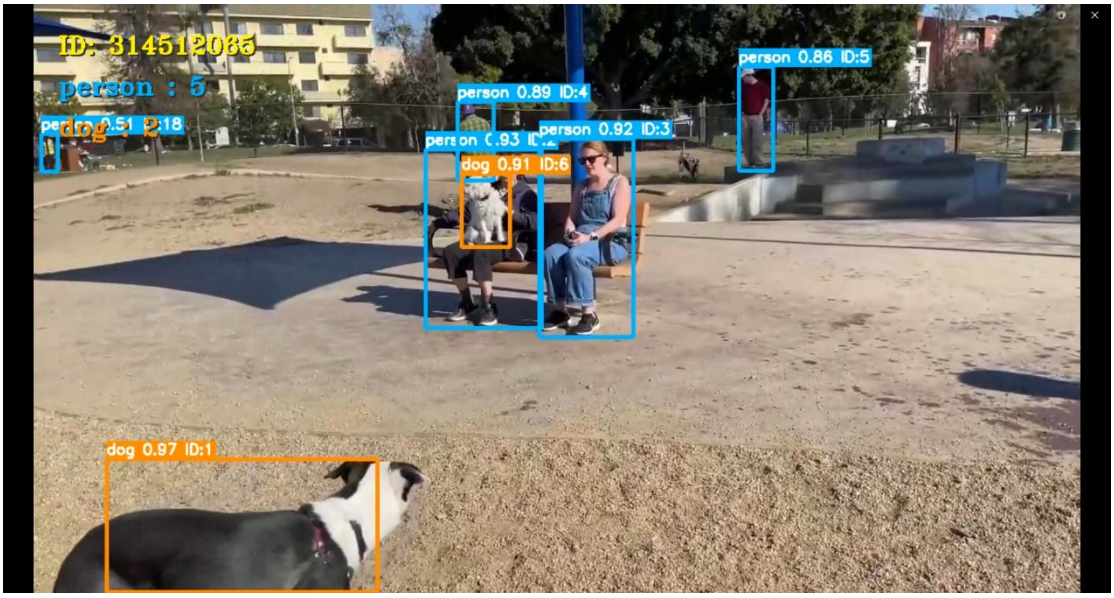


(3)

Baseline frame:



My video frame:



說明：(1) 與 (2) 的方法有效改善 Baseline 模型在部分影格中無法正確辨識狗的問題。由比較 (1)、(2) 與 (3) 各影格之偵測結果可知，經本方法處理後的影片，在人與狗兩類物件的辨識上，相較於 Baseline 具有更高的辨識準確度與穩定性。

修改部分:

提高輸入解析度 imgsz
調整信心值門檻 conf(降低錯誤檢測)
新增 IoU 門檻 iou（改善重疊框與重複偵測）
加入最小框面積過濾 MIN_BOX_AREA（降低假陽性、提升 precision）
加入 ByteTrack 追蹤（間接提升有效辨識準度）

```
IMGSZ = 1280
CONF = 0.35
IOU = 0.55
MIN_BOX_AREA = 900
TRACKER_YAML = "bytetrack.yaml"
```

```
results = model.track(
    frame,
    persist=True,
    tracker=TRACKER_YAML,
    classes=target_classes,
    conf=CONF,
    iou=IOU,
    imgsz=IMGSZ,
    verbose=False
)
```

```
if (x2 - x1) * (y2 - y1) < MIN_BOX_AREA:
    continue
```

透過提高模型輸入解析度，可在推論過程中保留更多影像細節，使模型對於尺寸較小或距離較遠的目標具有更佳的辨識能力，進而降低因特徵解析度不足所造成的漏檢問題。

同時，適度降低信心值門檻可保留更多潛在正確的偵測結果，並藉由搭配最小邊界框面積過濾機制，有效抑制因門檻降低所引入的假陽性。另透過調整 Non-Maximum Suppression（NMS）的 IoU 門檻，可合併同一目標的重疊候選框，減少重複偵測並提升定位穩定度。

最後，結合 ByteTrack 進行跨影格追蹤，雖不改變單幀偵測結果，但可抑制短暫誤檢與框抖動，使目標在時間軸上呈現連續且穩定的偵測結果，進一步提升整體辨識的可靠性與實用性。